Lab 21 — Web Application Firewall with ModSecurity and the OWASP CRS

> **SY0-701 Objective 3.2** — Given a scenario, apply security principles to secure enterprise infrastructure
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~25 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Deploy a layer 7 web application firewall, baseline it in detection-only mode, switch it to blocking, watch it stop SQL injection, XSS, path traversal and command injection, and tune one false positive narrowly.

You'll build: An Apache site protected by ModSecurity 2 and the OWASP Core Rule Set (CRS), with logged rule IDs and a scoped rule exclusion.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 20 (layer 4 firewall) is useful context; tools installed in step 1.

> **Legal note:** Attack payloads are sent only to `127.0.0.1` inside your own playground. Sending them to systems you do not own or have written permission to test is an offence under the Singapore Computer Misuse Act 1993.

Background

- A **WAF** is a **layer 7** firewall for HTTP. It reads the URL, headers, cookies and body after TLS is terminated — content a layer 4 firewall never sees.
- The OWASP **CRS** uses a **negative security model** (block known-bad patterns) with **anomaly scoring**: each matching rule adds points and the request is blocked when the inbound score reaches the threshold (5 by default).
- Always start in **DetectionOnly** mode, review what would be blocked, tune out false positives, then switch `SecRuleEngine On`.
- Tuning must be **narrow**: exclude one rule group for one parameter on one path, never switch off the WAF for the whole site.
- Placement and failure mode: a WAF usually sits inline in front of the web tier (often in a reverse proxy) and should normally **fail closed** for sensitive applications.

Steps

1. Install Apache, ModSecurity and the OWASP CRS

Port 80 is freed first in case nginx from another lab is running.

```
systemctl stop nginx 2>/dev/null || true
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y apache2
libapache2-mod-security2 modsecurity-crs curl
a2enmod security2 >/dev/null
systemctl restart apache2
apachectl -M 2>/dev/null | grep security2
```

2. Create the ModSecurity configuration in DetectionOnly mode

Copy the recommended config (its default is `DetectionOnly`) and make sure the CRS is loaded exactly once.

```
cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
grep -n '^SecRuleEngine' /etc/modsecurity/modsecurity.conf
grep -q 'modsecurity-crs' /etc/apache2/mods-enabled/security2.conf \
  && echo "CRS already included by security2.conf" \
  || echo 'IncludeOptional /usr/share/modsecurity-crs/*.load' > /etc/modsecurity/zz-load-
crs.conf
apachectl configtest && systemctl restart apache2
ls /usr/share/modsecurity-crs/rules/*.conf | wc -l
```

3. Send an attack in detection-only mode and read the warning

The request still succeeds (HTTP 200), but ModSecurity logs which rules would have blocked it.

```
curl -s -o /dev/null -w "DetectionOnly SQLi -> HTTP %{http_code}\n" -G --data-urlencode
"id=1' OR '1'='1" http://127.0.0.1/
sleep 1
grep 'ModSecurity: Warning' /var/log/apache2/error.log | grep -o 'id "[0-9]*"' | sort |
uniq -c
```

4. Switch the rule engine to blocking mode

```
sed -i 's/^SecRuleEngine DetectionOnly/SecRuleEngine On/'
/etc/modsecurity/modsecurity.conf
grep '^SecRuleEngine' /etc/modsecurity/modsecurity.conf
apachectl configtest && systemctl restart apache2
```

5. Fire SQLi, XSS, traversal and command-injection payloads

Each attack should now return `403 Forbidden`, while a normal search still returns `200`.

```
for p in "id=1' OR '1'='1" "q=<script>alert(1)</script>" "file=../../../../etc/passwd"
"cmd=;cat /etc/passwd"; do
  printf '%-32s -> ' "$p"
  curl -s -o /dev/null -w "%{http_code}\n" -G --data-urlencode "$p" http://127.0.0.1/
done
curl -s -o /dev/null -w "benign search                    -> %{http_code}\n"
"http://127.0.0.1/?q=security+plus"
```

```
id=1' OR '1'='1                 -> 403
q=<script>alert(1)</script>     -> 403
```

```
file=../../../../etc/passwd      -> 403
cmd=;cat /etc/passwd             -> 403
benign search                    -> 200
```

6. Read the matched rule IDs in the logs

The error log names every rule that matched; 949110 is the CRS rule that blocks once the anomaly score is exceeded.

```
grep -c 'Access denied with code 403' /var/log/apache2/error.log
grep 'ModSecurity' /var/log/apache2/error.log | grep -o 'id "[0-9]*"' | sort | uniq -c |
sort -rn | head -12
grep 'Access denied' /var/log/apache2/error.log | tail -1 | grep -o 'msg "[^"]*"'
```

Rule families: 942xxx SQL injection, 941xxx XSS, 930xxx path traversal / local file inclusion, 932xxx remote command execution. 920350 is a protocol-enforcement warning (the Host header is a bare IP address) and 980130 is the end-of-request anomaly summary.

7. Tune a false positive with a narrow exclusion

A CMS editor at `/admin/edit` legitimately accepts HTML in the `body` field. Exclude the XSS rules for that one parameter on that one path only.

```
curl -s -o /dev/null -w "editor body, before tuning -> %{http_code}\n" -G --data-urlencode
"body=<script>alert(1)</script>" http://127.0.0.1/admin/edit
cat > /etc/modsecurity/zz-lab21-exclusions.conf <<'EOF'
# Lab 21: the CMS editor accepts HTML in ARGS:body on /admin/edit only
SecRule REQUEST_URI "@beginsWith /admin/edit" \
    "id:1000,phase:1,pass,nolog,ctl:ruleRemoveTargetByTag=attack-xss;ARGS:body"
EOF
apachectl configtest && systemctl reload apache2
sleep 1
curl -s -o /dev/null -w "editor body, after tuning  -> %{http_code}\n" -G --data-urlencode
"body=<script>alert(1)</script>" http://127.0.0.1/admin/edit
curl -s -o /dev/null -w "other field, same path     -> %{http_code}\n" -G --data-urlencode
"q=<script>alert(1)</script>" http://127.0.0.1/admin/edit
curl -s -o /dev/null -w "same field, other path     -> %{http_code}\n" -G --data-urlencode
"body=<script>alert(1)</script>" http://127.0.0.1/
```

```
editor body, before tuning -> 403
editor body, after tuning  -> 404
other field, same path     -> 403
same field, other path     -> 403
```

`404` means the WAF let the request through and Apache simply has no `/admin/edit` page — the exclusion is scoped to one field on one path.

Test it

```
curl -s -o /dev/null -w "%{http_code}\n" -G --data-urlencode "id=1' UNION SELECT
username,password FROM users--" http://127.0.0.1/
```

```
curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1/"
grep '^SecRuleEngine' /etc/modsecurity/modsecurity.conf
```

Expected: `403` for the UNION-based SQL injection, `200` for the normal page, and `SecRuleEngine On`.

Troubleshooting

Symptom	Fix
`Found another rule with the same id` on configtest	The CRS is included twice; remove the duplicate include (keep only the one in `security2.conf`).
Apache fails to start: `Address already in use`	Another web server holds port 80; run `systemctl stop nginx` and restart Apache.
An attack returns 200 after step 4	Check `SecRuleEngine On` is set and Apache was restarted; confirm the CRS rules count in step 2 is greater than 0.
Step 7 still returns 403 after tuning	Another non-XSS rule also matched; read its ID in the error log and decide whether a second narrow exclusion is justified.

Challenge

Raise the CRS paranoia level to 2 in `/etc/modsecurity/crs/crs-setup.conf` (rule 900000, `tx.paranoia_level=2`), restart Apache and find one benign request that becomes a false positive. Write the narrowest exclusion that fixes it and explain the trade-off.

Reflection

Lab 20's layer 4 firewall passed the same SQL injection that the WAF blocked. Explain why, and decide whether this WAF should fail open or fail closed for an online banking site.

Exam tips

- A WAF protects web applications at layer 7 (SQLi, XSS, traversal); it does not replace secure coding or patching.
- Negative model = block known-bad (CRS); positive model = allow only known-good (stronger, harder to maintain).
- Deploy in detection-only first, tune false positives, then enforce.
- WAF, NGFW and UTM are all exam firewall types — WAF is the only one dedicated to HTTP applications.

References

- OWASP Core Rule Set documentation — https://coreruleset.org/docs/
- OWASP Top 10 — https://owasp.org/Top10/
- ModSecurity 2 reference manual — https://github.com/owasp-modsecurity/ModSecurity/wiki/Reference-Manual-(v2.x)
- MITRE ATT&CK T1190 Exploit Public-Facing Application — https://attack.mitre.org/techniques/T1190/